

# Mushroom Classification – Variable Cheat-Sheet

Hand-off note for the team. Use these variables to build more models without re-reading the code.

## 1. The key thing for modeling

Models take the fully preprocessed matrix (scaled + one-hot, no NaN) plus the target:

```
model.fit(X_train_prep, y_train)
model.predict(X_test_prep)
model.score(X_test_prep, y_test)
```

| Variable                                | What it holds                                                    | Old name                                          |
|-----------------------------------------|------------------------------------------------------------------|---------------------------------------------------|
| <b>X_train_prep / X_test_prep</b>       | model-ready: StandardScaler numbers + one-hot categories, no NaN | old X_train / X_test (but get_dummies + unscaled) |
| <b>X_train_robprep / X_test_robprep</b> | same, but with RobustScaler (only if you want to test robust)    | (new)                                             |
| <b>y_train / y_test</b>                 | target: 0 = edible, 1 = poisonous                                | same (source was ys)                              |

Default = X\_train\_prep (Standard ~ Robust; Standard marginally ahead).

## 2. Gotcha – X\_train now means something different!

| Variable                | What it holds NOW                                        | Old name               |
|-------------------------|----------------------------------------------------------|------------------------|
| <b>X / y</b>            | raw features / target from the original df               | used to be Xs / ys     |
| <b>X_train / X_test</b> | RAW: categories as text + NaN (we split BEFORE encoding) | same name, new content |

Old code that fed X\_train straight into a model now breaks (strings + NaN). Replace X\_train -> X\_train\_prep, X\_test -> X\_test\_prep.

## 3. For EDA (readable, training data only)

| Variable           | What it holds                                                        | Old name    |
|--------------------|----------------------------------------------------------------------|-------------|
| <b>df_train</b>    | training rows: readable categories + class + NaN -> patterns / plots | (new)       |
| <b>df_metr_nan</b> | numbers 0/1/2 only, with NaN                                         | df_metrical |

Do EDA only on df\_train. Never on df (contains test) or X\_test (the vault).

## 4. Intermediate steps (if you build your own preprocessing)

| Variable                                   | What it holds                                           |
|--------------------------------------------|---------------------------------------------------------|
| <b>num_column / categ_column</b>           | column lists: ['0','1','2'] / the 5 categorical columns |
| <b>X_train_median / X_test_median</b>      | numbers after median-impute (still unscaled)            |
| <b>X_train_scaled / X_train_rob_scaled</b> | numbers impute + Standard / Robust scaler               |
| <b>X_train_onehot / X_test_onehot</b>      | categories (imputed 'MISSING') one-hot encoded          |

How the model matrix is built: X\_train\_prep = np.hstack([X\_train\_scaled, X\_train\_onehot])

## 5. Fitted transformers (to treat NEW data the same way)

scaler (StandardScaler), rob\_scaler (RobustScaler), onehot (OneHotEncoder).

Note: imputer is used twice (median for numbers, then 'MISSING' for categories) - it ends up pointing at the category imputer.

## 6. Models already built

logreg (LogReg), knn (kNN + StandardScaler), knn2 (kNN + RobustScaler).